

Project Title : AI Regression Test Selection Agent using Langflow

1. Introduction

Project Overview

The AI Regression Test Selection Agent is an intelligent QA automation solution developed using **Langflow** and **Large Language Models (LLMs)**.

The system automatically analyzes release changes and identifies the most relevant regression test scripts that should be executed for a release.

The primary goal of the solution is to reduce the significant manual effort involved in regression planning and improve software release efficiency through intelligent automation.

2. Problem Statement

Current Challenges in Regression Testing

In modern Agile and DevOps environments, software applications undergo continuous:

- Feature enhancements
- Bug fixes
- Code changes
- Configuration updates

Due to frequent releases, executing the complete regression suite for every build becomes:

- Time-consuming
- Expensive
- Inefficient

Challenges Faced by QA Teams

QA teams face several problems during regression testing:

- Large regression suites with thousands of automated scripts
- Limited testing windows before release deadlines
- Difficulty identifying impacted functionalities
- Redundant execution of unrelated test cases
- Missed critical validations due to poor test selection
- High infrastructure and execution costs

Current Manual Process

Currently, regression test selection depends heavily on manual analysis and QA experience.

This results in:

- Increased release risk
- Longer testing cycles
- Poor regression optimization
- Inefficient resource utilization

3. Manual Effort and Identified Pain Point

Major Pain Point

Manual Regression Test Selection

QA engineers spend significant manual effort performing the following tasks:

- Reviewing release notes
- Understanding impacted modules
- Identifying relevant regression scripts
- Removing unrelated scripts
- Preparing execution plans
- Prioritizing testing areas

Why This Is a Problem

This process is:

- Repetitive
- Time-consuming
- Error-prone
- Experience-dependent

As release frequency increases, manual regression analysis becomes difficult to scale efficiently.

4. Proposed AI Solution

AI Regression Test Selection Agent

The proposed AI-powered solution uses:

- Langflow
- Large Language Models (LLMs)
- Intelligent workflow orchestration

to automate regression test selection.

What the System Does

The AI system:

Analyzes

- Release notes
- Change requests
- Feature updates
- Bug fixes

Identifies

- Impacted modules
- Related functionalities
- Critical risk areas

Automatically Determines

- Which regression scripts should run
- Which scripts can be skipped
- Which areas require additional validation

5. Objective

The objective of the solution is to:

- Reduce manual regression analysis effort
- Improve release confidence
- Optimize regression execution
- Accelerate testing cycles
- Improve QA productivity
- Provide AI-driven regression recommendations

6. Example Scenario

Release 7.1 Updates

The following release changes are introduced:

- MFA authentication added
- Payment retry logic modified
- Cart page redesigned
- Session timeout handling improved

7. Existing Regression Suite

Script Name	Covered Module
test_login.py	Login
test_payment.py	Payment
test_cart_ui.py	Cart
test_profile.py	UserProfile
test_session_timeout.py	Session

8. Expected AI Output

Selected Regression Scripts

- test_login.py
- test_payment.py
- test_cart_ui.py
- test_session_timeout.py

Excluded Scripts

- test_profile.py

Risk Areas

Module	Risk Level
Authentication	Critical
Payment	High

9. Langflow Workflow Architecture

High-Level Workflow

Release Notes Input



Prompt Template



LLM Node



Regression Script Mapper



Optimized Regression Output

10. Langflow Workflow Description

Step 1 — Input Collection

The system accepts:

- Release notes
- Change requests
- Regression suite metadata

Step 2 — Prompt Engineering

The Prompt Template structures the release information for AI analysis.

Step 3 — LLM Analysis

The LLM:

- Understands release changes
- Identifies impacted functionalities
- Maps release changes with regression scripts

Step 4 — Regression Optimization

The system:

- Selects relevant scripts
- Excludes unnecessary scripts
- Highlights risk areas

11. Langflow Components

Component	Purpose
Input Node	Accept release changes
Prompt Template	Prepare AI instructions
LLM Node	Analyze release impact
Script Mapper	Match scripts with modules
Output Node	Generate optimized regression plan

12. Prompt Template

You are a Senior QA Regression Architect.

Analyze the release changes and optimize the regression suite.

Tasks:

- 1. Identify impacted modules
- 2. Select relevant regression scripts
- 3. Exclude unrelated scripts
- 4. Highlight risk areas

Release Changes:

{release_input}

Regression Suite:

{suite_input}

13. Technologies Used

Component	Technology
Workflow Orchestration	Langflow
LLM	GPT-4o / Claude / Gemini / Llama 3
Backend	Python
Input Format	Release Notes / Metadata
Output Format	JSON / Text Report
Domain	QA Automation

14. Business Benefits

Engineering Benefits

- Reduced manual regression effort
- Faster regression planning
- Improved testing efficiency
- Better release validation

Management Benefits

- Reduced testing cost
- Faster software releases
- Improved release confidence
- Better QA productivity

15. Future Enhancements

Future versions may include:

- Dependency impact analysis
- Historical execution intelligence
- Risk scoring engine
- CI/CD integration
- Vector database support
- Autonomous regression planning

16. Conclusion

The AI Regression Test Selection Agent demonstrates how **Generative AI** and **Langflow** can simplify and automate regression planning activities.

The solution:

- Reduces manual effort
- Improves regression efficiency
- Optimizes test execution
- Accelerates software releases
- Provides intelligent QA recommendations

This prototype establishes a strong foundation for building future AI-powered QA automation agents.